Reinforcement Learning

Exam Master Summary

Stefan Eder

Based on Sutton & Barto (2nd ed.)

This document is a **compact exam summary** of the definitions and core concepts needed to answer typical past-exam questions. It is **not** meant to replace reading the book or attending the lecture. The guiding principle is: **brief and dense**, focusing on the minimum theory and notation you must be fluent in during the exam. In each chapter, only 3 past-exam questions are included for brevity. For full practice, use the master collection of old exam questions.

Contents

1 Finite Markov Decision Processes **1**
1.1 Preliminaries (exam quick): learning paradigms . . . 1
1.2 The agent–environment interface . . . 1
1.3 Markov property and MDP dynamics . . . 2
1.4 Goals, rewards, and the reward hypothesis . . . 2
1.5 Returns and episodes . . . 3
1.6 Policies and value functions . . . 4
1.7 Optimal value functions and Bellman optimality . . . 4
1.8 Past-exam questions . . . 5

2 Multi-Armed Bandits **6**
2.1 Problem setup and notation . . . 6
2.2 Exploration vs exploitation . . . 6
2.3 Action-value methods . . . 6
2.4 ε-greedy . . . 7
2.5 Optimistic initial values . . . 8
2.6 UCB (Upper Confidence Bound) . . . 8
2.7 Non-stationary problems . . . 9
2.8 Gradient bandits . . . 9
2.9 Past-exam questions . . . 10

3 Dynamic Programming **12**
3.1 What DP assumes (planning with a model) . . . 12
3.2 Iterative policy evaluation . . . 12
3.3 Policy improvement . . . 13
3.4 Policy iteration . . . 13
3.5 Value iteration . . . 14
3.6 Asynchronous DP (high-level) . . . 14
3.7 Past-exam questions . . . 14
3.8 Typical exam pitfalls . . . 15

4 Monte Carlo Methods **16**
4.1 Core idea: learn from complete episodes (no model, no bootstrapping) . . . 16
4.2 MC prediction: estimating v_π (or q_π) for a fixed policy . . . 16
4.3 MC control: improving the policy . . . 17
4.3.1 Why MC control usually estimates q_π . . . 17
4.3.2 Exploring starts . . . 17
4.3.3 On-policy MC control for ε-soft policies . . . 18
4.4 Off-policy MC prediction/control and importance sampling . . . 18
4.5 Past-exam questions . . . 20
4.6 Typical exam pitfalls . . . 20

5 Temporal Difference Learning **21**
5.1 Core idea: bootstrapping from experience (model-free) . . . 21
5.2 TD(0) prediction . . . 21
5.3 TD control: learning action values . . . 22
5.3.1 SARSA (on-policy TD control) . . . 22
5.3.2 Off-policy SARSA and importance sampling (exam quick) . . . 23
5.3.3 Expected SARSA (on-policy, lower variance target) . . . 23

5.3.4 Q-learning (off-policy TD control) 23
5.4 n-step TD and the TD–MC spectrum 25
5.5 Eligibility traces and TD(λ) 25
5.5.1 Batch learning and function approximation 26
5.6 Past-exam questions 26
5.7 Typical exam pitfalls 26

6 Policy Gradient Methods **27**
6.1 Why policy gradients 27
6.2 REINFORCE 27
6.3 Baselines and variance reduction 28
6.4 Actor–critic 29
6.4.1 (Very brief) PPO 29
6.5 Past-exam questions 30
6.6 Typical exam pitfalls 30

Chapter 1

Finite Markov Decision Processes

1.1 Preliminaries (exam quick): learning paradigms

Definition

Supervised learning: learn a mapping $x \mapsto y$ from labeled input–output pairs (x_i, y_i) (classification/regression).
Unsupervised learning: discover structure in unlabeled data x_i (clustering, dimensionality reduction).
Reinforcement learning (RL): learn how an **agent** should act in an **environment** to maximize cumulative reward via interaction.

Definition

Agent: the decision maker selecting actions according to a policy.
Environment: everything outside the agent; produces next state and reward in response to the agent's action.

Example

Typical mapping: image classification $\to$ supervised; clustering songs $\to$ unsupervised; playing chess / control $\to$ RL.
The exploration–exploitation trade-off is characteristic for RL/bandits due to interaction; it is not the standard setting in supervised/unsupervised learning.

1.2 The agent–environment interface

Definition

At each discrete time step $t = 0, 1, 2, \ldots$ the agent observes a *state* $S_t \in \mathcal{S}$ and selects an *action* $A_t \in \mathcal{A}(S_t)$. The environment responds with a *reward* $R_{t+1} \in \mathbb{R}$ and a next state S_{t+1}. The sequence

$$S_0, A_0, R_1, S_1, A_1, R_2, \ldots$$

is called a *trajectory* (or history).

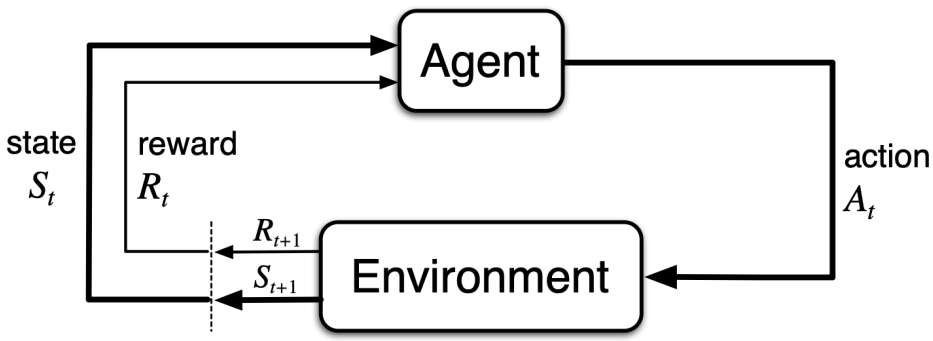

Figure 1.1: Standard agent–environment interaction at time t.

Definition

(**Policy**) A *policy* π specifies the agent's behavior. In the most general (stochastic) case,

$$\pi(a \mid s) \;=\; \Pr(A_t = a \mid S_t = s).$$

A policy is *deterministic* if it selects a single action with probability 1 in each state.

1.3 Markov property and MDP dynamics

Definition

(**Markov property**) A state signal has the Markov property if the conditional distribution of the next state and reward depends on the past only through the present state (and action):

$$\Pr(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a, \text{past}) \;=\; \Pr(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a).$$

Example

If the state is "current chess board position", then the next position depends only on that position and your move, not on the earlier move history. If the state were instead "current square of a knight" (but not the full board), then it is *not* Markov because the hidden board matters.

Definition

Stochastic process: a collection of random variables indexed by time, e.g. $\{S_t\}_{t\geq 0}$.
State-space cardinality: How many *different situations* / state spaces the agent might face.
Discrete state space: Countably many states (e.g. chess positions)
Continuous state space: Uncountably many states (e.g. real-valued sensor readings)

Definition

A **finite Markov decision process (finite MDP)** is defined by finite state and action sets and the *dynamics*

$$p(s', r \mid s, a) \;=\; \Pr(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a).$$

Often we also use the expected immediate reward

$$r(s, a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a], \quad \text{and} \quad p(s' \mid s, a) = \Pr(S_{t+1} = s' \mid S_t = s, A_t = a).$$

In words: it's the environment's rulebook: "if you do a in s, what tends to happen next?"

1.4 Goals, rewards, and the reward hypothesis

Definition

The **reward signal** is the immediate scalar feedback from the environment. The **goal** of the agent is to maximize expected long-term reward, formalized via the *return* (next section).

Example

A reward is *not* the same as "winning" or "performance"—it is a design choice. For chess: $+1$ for win, 0 draw, -1 loss is common. For navigation: -1 per time step and 0 at the goal encourages shortest paths.

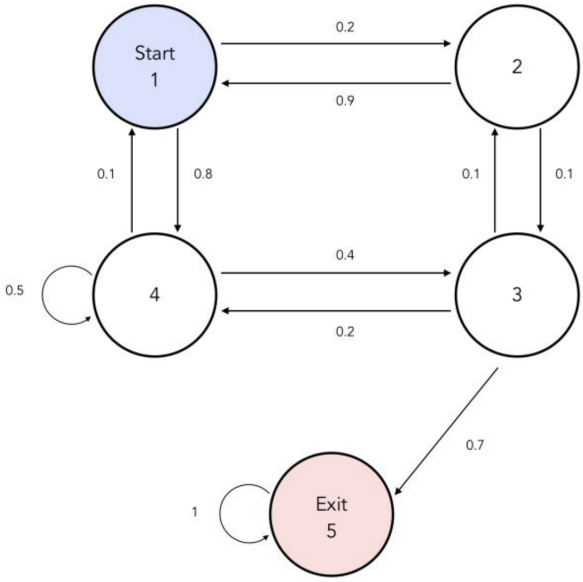

Figure 1.2: A small MDP can be visualized as a directed graph with action-labeled transitions.

1.5 Returns and episodes

Definition

(**Episodic tasks**) An episode terminates in a *terminal state* at time T. The **return** from time t is

$$G_t \;=\; R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \;=\; \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1},$$

where $\gamma \in [0, 1]$ is the **discount factor**. Common special case: *undiscounted episodic return* with $\gamma = 1$.

Definition

(**Continuing tasks**) There is no terminal time T. Discounting ($\gamma < 1$) is a standard way to keep the infinite sum well-behaved and to express preference for sooner rewards.

Example

(Continuing task, constant reward) If $R_{t+1} = r$ for all t and $0 \leq \gamma < 1$, then

$$G_t = r + \gamma r + \gamma^2 r + \cdots = \frac{r}{1-\gamma}.$$

Example

(**Backward computation trick**) Because

$$G_t = R_{t+1} + \gamma G_{t+1},$$

you can compute returns efficiently by starting from the end of an episode and working backward.

1.6 Policies and value functions

Definition

Given a policy π, the **state-value function** and **action-value function** are

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s], \qquad q_\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

Intuition: $v_\pi(s)$ is how good it is to be in s under π; $q_\pi(s,a)$ is how good it is to take a in s and then follow π.

Definition

(**Bellman expectation equation for v_π**) For a finite MDP,

$$v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\Big[r + \gamma v_\pi(s')\Big].$$

This expresses "value = expected one-step reward + discounted value of successor state".

Definition

(**Bellman expectation equation for q_π**)

$$q_\pi(s,a) = \sum_{s',r} p(s', r \mid s, a)\Big[r + \gamma \underbrace{\sum_{a'} \pi(a' \mid s')\, q_\pi(s', a')}_{= v_\pi(s')}\Big].$$

1.7 Optimal value functions and Bellman optimality

Definition

The **optimal state-value function** and **optimal action-value function** are

$$v_*(s) = \max_\pi v_\pi(s), \qquad q_*(s,a) = \max_\pi q_\pi(s,a).$$

An **optimal policy** π_* attains these maxima.

Definition

(**Bellman optimality equation for v_***)

$$v_*(s) = \max_a \sum_{s',r} p(s', r \mid s, a)\Big[r + \gamma v_*(s')\Big].$$

Definition

(**Bellman optimality equation for q_***)

$$q_*(s,a) = \sum_{s',r} p(s', r \mid s, a)\Big[r + \gamma \max_{a'} q_*(s', a')\Big].$$

A greedy policy w.r.t. q_* is optimal:

$$\pi_*(s) \in \arg\max_a q_*(s,a).$$

1.8 Past-exam questions

Exam-style question

Match each definition with the concept:

- A. Defines the agent's way of behaving at a given time
- B. A model that mimics the behavior of the environment
- C. A numerical value that defines good/bad events; provided each time step
- D. Indicates long-term desirability of states; expected sum of future rewards

Concepts: (1) policy, (2) model of the environment, (3) reward signal, (4) value function.

Key idea / answer sketch

A $\to$ policy B $\to$ model of the environment C $\to$ reward signal D $\to$ value function.

Exam-style question

Suppose $\gamma = \frac{1}{2}$ and rewards $R_1 = -7, R_2 = 6, R_3 = 1, R_4 = 5, R_5 = 2$ with terminal time $T = 5$ (and $R_t = 0$ for $t > T$). Compute $G_0, \ldots, G_5$.

Key idea / answer sketch

Work backward using $G_t = R_{t+1} + \gamma G_{t+1}$ and $G_5 = 0$:

$$G_4 = 2, \quad G_3 = 5 + \tfrac{1}{2} \cdot 2 = 6, \quad G_2 = 1 + \tfrac{1}{2} \cdot 6 = 4,$$

$$G_1 = 6 + \tfrac{1}{2} \cdot 4 = 8, \quad G_0 = -7 + \tfrac{1}{2} \cdot 8 = -3, \quad G_5 = 0.$$

Exam-style question

("It is better to use the undiscounted return ($\gamma = 1$) for non-episodic tasks." True or false?

Key idea / answer sketch

False in general. For continuing tasks, an undiscounted infinite sum can diverge (or be ill-defined). Typical fixes: discounted formulation with $\gamma < 1$ (and bounded rewards), or an average-reward objective.

Multi-Armed Bandits

2.1 Problem setup and notation

Definition

A k-**armed bandit** has actions (arms) $a \in \{1, \ldots, k\}$. At each time step t the agent selects A_t and observes a scalar reward R_t (often written R_{t+1} in MDP notation).

Definition

(**True action value**) The (stationary) **true value** of arm a is

$$q_*(a) \;=\; \mathbb{E}[R_t \mid A_t = a].$$

The agent maintains an **estimate** $Q_t(a)$ and a **count** $N_t(a)$ = number of times arm a was selected up to t.

Definition

(**Stationary vs non-stationary**) Stationary means $q_*(a)$ does not change over time. Non-stationary means the reward distribution (hence $q_*(a)$) changes.

2.2 Exploration vs exploitation

Definition

Exploitation selects the action with the highest current estimate (greedy w.r.t. Q_t). **Exploration** selects actions to improve information/estimates, potentially sacrificing immediate reward.

2.3 Action-value methods

Key Formula / Theorem

(**Sample-average estimate**) If arm a has been selected $N_t(a) \geq 1$ times with observed rewards $R^{(1)}(a), \ldots, R^{(N_t(a))}(a)$, then

$$Q_t(a) = \frac{1}{N_t(a)} \sum_{i=1}^{N_t(a)} R^{(i)}(a).$$

Key Formula / Theorem

(**Incremental update**) For the chosen action A_t:

$$Q_{t+1}(A_t) = Q_t(A_t) + \alpha_t(R_t - Q_t(A_t)), \qquad Q_{t+1}(a) = Q_t(a) \;\; (a \neq A_t).$$

Constant $\alpha \in (0, 1]$ yields an **exponentially recency-weighted** average (useful for non-stationary problems).

Key Formula / Theorem

(**ISAM**) The **Incremental Sample-Averaging Method (ISAM)** uses step size $\alpha_t = \frac{1}{N_{t+1}(A_t)}$:

$$Q_{t+1}(A_t) = Q_t(A_t) + \frac{1}{N_{t+1}(A_t)}(R_t - Q_t(A_t)).$$

Same result as the sample-average estimate but is computationally far more efficient.

Example

If $Q_t(a) = -3$, $R_t = 2$, and $\alpha = 0.1$, then

$$Q_{t+1}(a) = -3 + 0.1(2 - (-3)) = -2.5.$$

2.4 ε-greedy

Definition

(**ε-greedy**) With probability $1 - \varepsilon$ choose a greedy action

$$A_t \in \arg\max_a Q_t(a),$$

and with probability ε choose an action uniformly at random from $\{1, \ldots, k\}$.

Note

(**Asymptotics**) With constant $\varepsilon > 0$, exploration never stops; a fraction ε of actions remains random forever. Smaller ε typically yields higher *asymptotic* reward but explores less. **GLIE** schedules make $\varepsilon \to 0$ while still exploring infinitely often. Pure greedy ($\varepsilon = 0$) can get stuck on a suboptimal arm due to early noise because it stops exploring.

Definition

(**GLIE**) **Greedy in the Limit with Infinite Exploration**: exploration persists forever but vanishes asymptotically, e.g. $\varepsilon_k = 1/k$ for episode k (one common schedule).

Example

Probability that the greedy action is selected under ε-greedy (uniform exploration):

$$\Pr(\text{greedy}) = (1 - \varepsilon) + \varepsilon \cdot \frac{1}{k}.$$

For $k = 10, \varepsilon = 0.7$: $\Pr(\text{greedy}) = 0.3 + 0.7 \cdot 0.1 = 0.37$.

Note

Under ε-greedy: if a *non-greedy* action is selected, then the ε-case definitely occurred. If the greedy action is selected, it could have come from either the greedy case or the ε-case.

2.5 Optimistic initial values

Intuition

(**Optimistic initialization**) Set initial estimates $Q_1(a)$ to values higher than plausible rewards. With greedy action selection, this encourages trying many actions early (untried actions stay "optimistic"). Works best in **stationary** problems; in non-stationary settings the initial optimism gets "washed out". Once values converge, the agent stops exploring. If rewards change later (e.g. unattractive actions become attractive), the initial optimism offers no help for this new shift.

Note

Why optimistic initialization "requires prior knowledge". Optimistic initialization needs a *rough sense of reward scale* (an upper bound / plausible range) so that $Q_1(a)$ is meaningfully "high". You do *not* need to know the full reward distribution, but you must pick values that are optimistic relative to typical returns.

Note

If rewards are strictly negative, values like $Q_1(a) = 0$ are still *optimistic* (higher than any plausible mean reward).

Note

Exam nuance: "explicit exploration–exploitation hyperparameter".

- **ε-greedy** explicitly controls exploration with ε: larger $\varepsilon \Rightarrow$ more random actions.
- **UCB** explicitly controls exploration with c: larger $c \Rightarrow$ larger uncertainty bonus.
- **Gradient bandits** do *not* have an ε-like or c-like parameter that directly sets an exploration rate. They explore because they **sample** from a softmax policy. The step size α controls *learning speed* of preferences, not an "exploration probability". (A softmax *temperature* τ could be introduced to tune randomness, but it is not part of the standard algorithm as typically presented.)

2.6 UCB (Upper Confidence Bound)

Key Formula / Theorem

(**UCB action selection**) Choose

$$A_t \in \arg\max_a \left[Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right],$$

where $c > 0$ controls exploration. (In practice each action is selected at least once so $N_t(a) \geq 1$.)
The second term is an **uncertainty bonus**: it decreases as $N_t(a)$ grows.
If $c = 0$, UCB reduces to greedy action selection: $A_t \in \arg\max_a Q_t(a)$.

Note

Unselected actions in UCB. If an action a is not selected for a while, then $N_t(a)$ stays constant while $\ln t$ grows, so the bonus $c\sqrt{\ln t / N_t(a)}$ typically *increases* (slowly). Thus the uncertainty term for an unselected action does *not* decrease.

Intuition

Intuition (why the UCB bonus makes sense). UCB treats each action value estimate $Q_t(a)$ as "optimistic" by adding a *confidence bonus*

$$c\sqrt{\frac{\ln t}{N_t(a)}}.$$

This bonus is large when an arm is **uncertain** (small $N_t(a)$), so the algorithm is tempted to try it—even if its current mean $Q_t(a)$ is not the best yet. As you sample an arm more often, $N_t(a)$ grows and the bonus shrinks, meaning: **once you are confident an arm is bad, it stops looking attractive**.
So UCB explores *selectively*: it does not explore uniformly at random (like ε-greedy), but instead prioritizes arms where the current evidence is weak and a "surprise" could still happen. The $\ln t$ term slowly increases the pressure to occasionally re-check arms, but much less than the shrinkage from $N_t(a)$, so the focus shifts from exploration to exploitation as uncertainty vanishes.
In short: **explore what you are unsure about; exploit what you are confident is good.**

Note

(**Stationarity assumption**) The classic UCB rule is derived for (approximately) **stationary** bandits. In strongly non-stationary settings it can adapt poorly unless modified (e.g., sliding-window / discounted UCB).

2.7 Non-stationary problems

Intuition

For non-stationary bandits, **constant step sizes** (exponential recency weighting)

$$Q_{t+1}(A_t) = Q_t(A_t) + \alpha(R_t - Q_t(A_t))$$

help track changes, because older rewards get exponentially downweighted.

2.8 Gradient bandits

Definition

(**Preference parameters**) In gradient bandits we do *not* estimate action values $Q(a)$. Instead we maintain an unconstrained real-valued **preference** (or "logit")

$$H_t(a) \in \mathbb{R} \qquad \text{for each action } a.$$

Only *differences* between preferences matter: if you add the same constant to all $H_t(a)$, the resulting policy does not change. Large $H_t(a)$ means "the policy should be more likely to pick a", small $H_t(a)$ means less likely.
Initialization: typically set $H_1(a) = 0$ for all a (or small random values), then H is learned solely via the update rule.

Key Formula / Theorem

(**Softmax policy from preferences**)

$$\pi_t(a) = \frac{\exp(H_t(a))}{\sum_b \exp(H_t(b))}.$$

Actions are sampled from $\pi_t(\cdot)$ (not greedy).

Key Formula / Theorem

(**Gradient bandit update**) With step-size $\alpha > 0$ and baseline $\bar{R}_t$ (often the running average reward): for all actions a,

$$H_{t+1}(a) = \begin{cases} H_t(a) + \alpha\,(R_t - \bar{R}_t)\,(1 - \pi_t(a)) & \text{if } a = A_t, \\ H_t(a) - \alpha\,(R_t - \bar{R}_t)\,\pi_t(a) & \text{if } a \neq A_t. \end{cases}$$

Intuition

Intuition (what this is doing). π_t is a softmax "lottery"; $H_t(a)$ are the unnormalized scores (logits).
After picking A_t and observing R_t:

- If $R_t > \bar{R}_t$: increase $H(A_t)$ (make it more likely); slightly decrease others.
- If $R_t < \bar{R}_t$: decrease $H(A_t)$; slightly increase others.

The $(1 - \pi_t(A_t))$ and $\pi_t(a)$ factors scale the step: big changes when the action was unlikely (surprising), small changes when it was already likely. Net effect: **push up actions that beat the baseline, push down actions that don't.**

Note

Preferences are not value estimates: $H_t(a)$ define π_t via softmax; they are *not* $Q(a)$.
Baseline must be action-independent: to keep the update unbiased, the baseline must not depend on the chosen action. Thus using $Q_t(A_t)$ as a baseline is generally **not** valid (unless using an action-independent baseline such as $\sum_a \pi_t(a) Q_t(a)$).

2.9 Past-exam questions

Exam-style question

Optimistic initial values bias the agent toward initially exploring actions with higher estimated rewards. True/False?

Key idea / answer sketch

True. Untried actions keep optimistic estimates, so greedy selection tends to try them.

Exam-style question

UCB balances exploration/exploitation by adding an uncertainty term to each action value. True/False?

Key idea / answer sketch

True. That is the defining idea of UCB: pick $\arg\max_a [Q_t(a) + \text{bonus}(a)]$.

Exam-style question

In gradient bandits, actions are selected proportional to their action preference. True/False?

Key idea / answer sketch

True (in an old exam). Gradient bandits sample from a softmax:

$$\pi_t(a) = \frac{\exp(H_t(a))}{\sum_b \exp(H_t(b))} \quad \Longleftrightarrow \quad \pi_t(a) \propto \exp(H_t(a)).$$

Note

Exam wording nuance. Taken literally, "proportional to the preference" means $\pi_t(a) \propto H_t(a)$, which is **false**. The past-exam solution marked **True**, so "preference" is meant in the *softmax sense*: probabilities are proportional to $\exp(H_t(a))$, not to $H_t(a)$ itself.

Chapter 3

Dynamic Programming

3.1 What DP assumes (planning with a model)

Definition

Dynamic Programming (DP) in RL refers to a family of *planning* algorithms that compute value functions / optimal policies *given a known MDP model*:

$$p(s', r \mid s, a) \quad \text{(and typically } \gamma\text{)}.$$

DP methods perform **expected backups** (sums over next states and rewards), rather than learning from sampled experience.

- **Backup / update:** update *one* component $V(s)$ (one state) using a Bellman equation.
- **Sweep:** a pass that performs (at most) one backup for *each* state $s \in \mathcal{S}$ (order arbitrary).
- **Iterative evaluation iterations:** repeated *sweeps* of T^π until $V \approx v_\pi$.
- **Policy-iteration iterations:** one *evaluation phase* (many sweeps) + one *improvement step*.

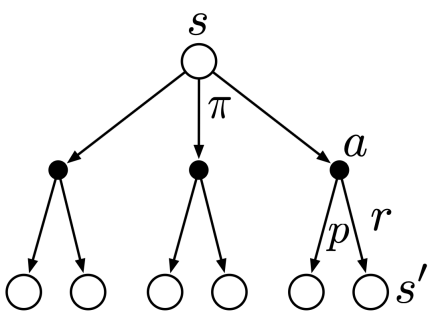

Figure 3.1: DP backup = expectation over all immediate successors (requires the model).

3.2 Iterative policy evaluation

Definition

(**Policy evaluation**) For a fixed policy π, compute v_π (or approximate it). Define the **Bellman expectation operator** T^π, which maps any candidate value function v to a new one-step lookahead value function:

$$(T^\pi v)(s) \;=\; \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma v(s')\big].$$

The true value function v_π is the **fixed point** of this operator:

$$v_\pi = T^\pi v_\pi \qquad \text{(i.e., applying } T^\pi \text{ does not change } v_\pi\text{)}.$$

Key Formula / Theorem

(**Synchronous / two-array iterative evaluation**) Initialize $V_0(s)$ and iterate

$$V_{k+1}(s) \;=\; (T^\pi V_k)(s) \;=\; \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V_k(s')\big].$$

Stop when $\max_s |V_{k+1}(s) - V_k(s)| < \theta$.

Note

(**In-place / one-array evaluation**) Update $V(s)$ immediately as you sweep states:

$$V(s) \leftarrow (T^\pi V)(s),$$

using the latest available values for successor states. This is often faster, and *the update order can matter*. **Unlike synchronous updates, later states in the same sweep may "see" earlier updated values.**

Note

DP evaluation also bootstraps. Iterative policy evaluation is a **bootstrapping** method: it updates $V(s)$ using the current estimates $V(\cdot)$ of successor states (e.g., $V_{k+1}(s)$ uses $V_k(s')$).

3.3 Policy improvement

Key Formula / Theorem

Given v_π, define

$$q_\pi(s, a) \;=\; \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma v_\pi(s')\big].$$

A **greedy improvement step** forms a new policy π' by

$$\pi'(s) \in \arg\max_a q_\pi(s, a) \quad \text{(ties: any tie-breaking is fine).}$$

Key Formula / Theorem

(**Policy improvement theorem**) If π' is greedy w.r.t. v_π (equivalently w.r.t. q_π), then

$$v_{\pi'}(s) \;\geq\; v_\pi(s) \quad \text{for all } s.$$

The improvement can be non-strict (sometimes $\pi' = \pi$).

3.4 Policy iteration

Note

(**Policy iteration**) Repeat:

1. **Evaluate** the current policy π (compute/approximate v_π).
2. **Improve** it: $\pi \leftarrow \text{Greedy}(q_\pi)$.

Stop when the policy is unchanged by improvement (*policy stable*). For finite MDPs (standard discounted setting), policy iteration reaches an optimal policy after a finite number of improvements.

Note

(**Modified policy iteration**) Do *partial* evaluation (e.g., a few sweeps) between improvements. This interpolates between policy iteration (full evaluation) and value iteration (minimal evaluation).

3.5 Value iteration

Key Formula / Theorem

(**Bellman optimality operator**)

$$(T^*v)(s) \;=\; \max_a \sum_{s',r} p(s',r \mid s,a)\big[r + \gamma v(s')\big].$$

The optimal value function satisfies $v_* = T^* v_*$ (unique fixed point under standard discounted assumptions).

Key Formula / Theorem

(**Value iteration**) Initialize V_0 and iterate

$$V_{k+1}(s) \;=\; (T^*V_k)(s) \;=\; \max_a \sum_{s',r} p(s',r \mid s,a)\big[r + \gamma V_k(s')\big].$$

After convergence, an optimal policy can be extracted greedily:

$$\pi_*(s) \in \arg\max_a \sum_{s',r} p(s',r \mid s,a)\big[r + \gamma V(s')\big].$$

Intuition

Interpretation: Value iteration can be seen as policy iteration where the evaluation step is truncated to *one* backup per sweep (i.e., it *implicitly* evaluates the greedy policy w.r.t. the current value estimate).

3.6 Asynchronous DP (high-level)

Definition

(**Asynchronous DP**) Updates states in any order, possibly skipping states:

$$V(s) \leftarrow (T^\pi V)(s) \quad \text{or} \quad V(s) \leftarrow (T^* V)(s).$$

Examples: in-place sweeps, prioritized sweeping (update states that are likely to change a lot).

3.7 Past-exam questions

Exam-style question

Given an arbitrary policy, the optimal policy can be computed directly by solving a system of linear equations using the Bellman equation. True/False?

Key idea / answer sketch

False. Linear Bellman equations solve v_π for a fixed π. Optimality involves a max (nonlinear) condition.

Exam-style question

Policy evaluation with reward -4 per step and $\gamma = 0.5$: value of state 0 after two sweeps (initial values 0, terminal value 0).

Key idea / answer sketch

Use the evaluation backup

$$V_{k+1}(s) = \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s, a)\Big[-4 + \gamma V_k(s')\Big].$$

With $V_0 \equiv 0$, the first sweep typically gives $V_1(s) = -4$ for nonterminal states that can transition in one step. Second sweep:

$$V_2(0) = -4 + 0.5 \cdot \mathbb{E}[V_1(S') \mid S = 0, \pi].$$

Note

Typical exam instantiation. If state 0 transitions to a terminal state with prob. 0.5 (value 0) and to a nonterminal state with prob. 0.5, then with $V_0 \equiv 0$: $V_1(0) = -4$ and $V_2(0) = -4 + 0.5 \cdot (0.5 \cdot 0 + 0.5 \cdot (-4)) = -5$.

Exam-style question

Policy improvement at state 0: compute $Q(\mathit{small})$, $Q(\mathit{large})$ and pick the greedy action.

Key idea / answer sketch

Here $R = -1$, $\gamma = 1$, and $Q(a) = \sum_{s'} p(s' \mid 0, a)\big[-1 + V(s')\big]$.

$Q(\mathit{small}) = 0.5(-1+3)+0.5(-1-1) = 0, \qquad Q(\mathit{large}) = 0.5(-1+5)+0.25(-1-3)+0.25(-1-7) = -1.$

Greedy choice: **small**.

3.8 Typical exam pitfalls

- Mixing up **evaluation** (compute v_π for fixed π) with **control** (improve π toward optimality).
- Forgetting DP is **model-based**: backups sum over all immediate successors using $p(s', r \mid s, a)$.
- Confusing **two-array** (order irrelevant) with **in-place** updates (order can matter a lot).
- "Greedy" in DP means **argmax**, not random exploration.
- Value iteration: convergence is typically **not finite**; stopping is approximate (threshold on Bellman residual / max update).

Chapter 4

Monte Carlo Methods

4.1 Core idea: learn from complete episodes (no model, no bootstrapping)

Definition

Monte Carlo (MC) methods estimate value functions (and improve policies) from **sampled episodes** of experience. They are **model-free** (no need for $p(s', r \mid s, a)$) and **do not bootstrap**: updates use empirical returns G_t, not $V(S_{t+1})$.

Note

MC typically assumes an **episodic** task (or at least well-defined episode segments), because you need G_t:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}.$$

4.2 MC prediction: estimating v_π (or q_π) for a fixed policy

Definition

(**First-visit MC prediction**) For each episode, for each state s, let $G(s)$ be the return following the *first time* s is visited in that episode. Estimate

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

by the sample average of observed first-visit returns:

$$V(s) \leftarrow \text{mean of returns observed for } s.$$

Note

(**First-visit MC prediction for action values**) To estimate $q_\pi(s, a)$, treat each (s, a) like a "state": for each episode, for the *first* time step t with $(S_t, A_t) = (s, a)$, compute the return G_t and update

$$Q(s, a) \leftarrow \text{mean of observed first-visit returns for } (s, a).$$

Definition

(**Every-visit MC prediction**) Same idea, but use the return following *every* visit to s in an episode. (You get more samples per episode, but they can be correlated within an episode.)

Note

First-visit vs every-visit: (in)dependence. For a fixed policy in an episodic Markov setting, the *first-visit return samples* for a given state (or state–action pair) across episodes are treated as i.i.d. draws from the same distribution (hence simple sample averaging works). In contrast, *every-visit* returns can be **correlated within an episode** (multiple visits share the same trajectory), so they are generally not i.i.d.

Key Formula / Theorem

(**Incremental mean update**) If $V(s)$ is the mean of n returns seen so far and a new sample return G arrives:

$$V(s) \leftarrow V(s) + \frac{1}{n+1}(G - V(s)).$$

Example

If $V(s) = 4$ based on $n = 3$ returns and the next return is $G = 1$, then

$$V(s) \leftarrow 4 + \tfrac{1}{4}(1 - 4) = 3.25.$$

4.3 MC control: improving the policy

4.3.1 Why MC control usually estimates q_π

Intuition

For control, it is convenient to estimate **action values**

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

because policy improvement becomes "make the policy greedy w.r.t. Q":

$$\pi(s) \in \arg\max_a Q(s, a).$$

4.3.2 Exploring starts

Definition

(**Exploring starts (ES)**) MC control with ES assumes that the start of an episode is randomized so that *every* state–action pair (s, a) can occur as (S_0, A_0) with nonzero probability:

$$\Pr(S_0 = s, A_0 = a) > 0 \quad \text{for all } (s, a).$$

Then, even if we do purely greedy improvement w.r.t. Q, we will still eventually obtain data for every (s, a), so $Q(s, a)$ can be learned everywhere and the greedy policy can become optimal. (Primarily a theoretical assumption; often unrealistic because you cannot choose arbitrary starts in real tasks.)

Intuition

Intuition (why ES is needed). Greedy improvement has a problem: if your policy never tries some action a in state s, then you will never see returns for (s, a), so $Q(s, a)$ never gets corrected. In the worst case you can get stuck with a suboptimal policy simply because some alternatives were never sampled.
Exploring starts is a clean "mathematical hack" that guarantees *coverage*: by forcing random initial (S_0, A_0), every state–action pair gets tried infinitely often (given enough episodes). Once that is guaranteed, greedy improvement is safe: eventually Q becomes accurate everywhere, and greedy w.r.t. Q can converge to an optimal policy.

4.3.3 On-policy MC control for ε-soft policies

Definition

(**ε-soft policies**) A policy π is **ε-soft** if it assigns *every* action in a state at least some minimum probability:

$$\pi(a \mid s) \;\geq\; \frac{\varepsilon}{|\mathcal{A}(s)|} \quad \text{for all } s \text{ and all } a \in \mathcal{A}(s),$$

with $\varepsilon > 0$. This guarantees ongoing exploration (no action is ever completely ruled out). A common way to construct an ε-soft policy is **ε-greedy** w.r.t. Q.

Intuition

Intuition (why ε-soft matters). Exploring starts (ES) injects exploration only at the *start* of episodes by assuming (S_0, A_0) can be any (s, a) with nonzero probability. Thus, across many episodes, every (s, a) gets tried (at least as a starting pair).
An ε-soft policy injects exploration *throughout* an episode: whenever a state s is visited, every action still has nonzero probability, even late in learning. This prevents "never tried $\Rightarrow$ never learned" and lets on-policy MC control explore without needing the ES assumption.

Note

(**On-policy MC control**) Repeatedly:

1. Generate an episode using the current ε-soft policy π.
2. Use returns to update $Q(s, a)$ (first-visit or every-visit).
3. Improve π to be ε-greedy w.r.t. the updated Q.

This couples evaluation + improvement (a form of generalized policy iteration).

4.4 Off-policy MC prediction/control and importance sampling

Definition

(**Off-policy**) Learn about a **target policy** π using episodes generated by a different **behavior policy** μ. Key requirement: **coverage** (support):

$$\mu(a \mid s) = 0 \implies \pi(a \mid s) = 0,$$

otherwise importance sampling ratios can be undefined (division by 0).

Intuition

Intuition: why we need importance sampling. Episodes from μ contain many action choices that π would make with different probabilities. Importance sampling reweights each sampled episode so that, *in expectation*, it counts as if it had been generated by π.
Rule of thumb: if an episode contains actions that are *unlikely under π but likely under μ*, it should count *less* (small weight). If it contains actions that are *likely under π but unlikely under μ*, it should count *more* (large weight).

Key Formula / Theorem

(**Importance sampling ratio = likelihood ratio**) For an episode segment from time t to terminal time T:

$$\rho_{t:T-1} \;=\; \prod_{k=t}^{T-1} \frac{\pi(A_k \mid S_k)}{\mu(A_k \mid S_k)}.$$

Interpretation: $\rho_{t:T-1}$ is the ratio

$$\frac{\Pr_\pi(A_t, \dots, A_{T-1} \mid S_t, \dots)}{\Pr_\mu(A_t, \dots, A_{T-1} \mid S_t, \dots)}.$$

Then $\rho_{t:T-1} G_t$ is an unbiased (but possibly high-variance) way to evaluate π using data from μ.

Intuition

Why the product can explode (variance intuition). The ratio multiplies many factors. Even if each factor is moderate, a long episode can make ρ very large or very small. So a few rare episodes can dominate the average $\Rightarrow$ huge variance. This is why off-policy MC is theoretically clean but can be numerically unstable.

Key Formula / Theorem

(**Ordinary IS = unbiased average of reweighted returns**) Estimate

$$v_\pi(s) \approx \frac{1}{n} \sum_{i=1}^{n} \rho^{(i)} G^{(i)}.$$

Unbiasedness identity:

$$\mathbb{E}_\mu[\rho\, G \mid S_t = s] \;=\; \mathbb{E}_\pi[G \mid S_t = s] \;=\; v_\pi(s).$$

But variance can be extremely large because a few samples with huge ρ can dominate.

Key Formula / Theorem

(**Weighted IS = normalize weights to tame variance**) Estimate

$$v_\pi(s) \approx \frac{\sum_{i=1}^{n} \rho^{(i)} G^{(i)}}{\sum_{i=1}^{n} \rho^{(i)}}.$$

This *normalizes* by total weight, so extremely large ρ values do not explode the estimate as easily. Tradeoff: biased for finite n, but typically much more stable. (As $n \to \infty$ the bias vanishes under standard conditions.)

Intuition

Ordinary vs weighted in one sentence. Ordinary IS is "estimate the expectation directly" (unbiased but noisy); Weighted IS is "take a weighted average of returns" (slight bias, much less noise).

Key Formula / Theorem

(**Incremental weighted-IS update (running weighted mean)**) Maintain cumulative weights $C(s)$ and update with new sample (ρ, G):

$$C(s) \leftarrow C(s) + \rho, \qquad V(s) \leftarrow V(s) + \frac{\rho}{C(s)}(G - V(s)).$$

This is just the online form of $V(s) = \frac{\sum_i \rho^{(i)} G^{(i)}}{\sum_i \rho^{(i)}}$. (Analogously for $Q(s,a)$ in MC control.)

4.5 Past-exam questions

Exam-style question

Match statements to methods:

- A: $\varepsilon_k = 1/k$ for the k-th episode.
- B: greedy action chosen with probability $1 - \varepsilon$ (constant ε).
- C: Learn about target policy π from experience generated by behavior policy μ.

Methods: GLIE MC control; on-policy MC control for ε-soft policies; off-policy learning.

Key idea / answer sketch

A $\rightarrow$ GLIE MC control B $\rightarrow$ on-policy MC control for ε-soft policies C $\rightarrow$ off-policy learning.

Exam-style question

Monte Carlo control methods compute an estimate of the state-value function v_π. True/False?

Key idea / answer sketch

False (as stated). MC control is typically formulated via estimating **action values** $q_\pi(s,a)$ (or q_*) to enable greedy improvement.

Exam-style question

In off-policy MC prediction, what condition is needed so importance sampling is well-defined?

Key idea / answer sketch

Coverage/support: if $\pi(a \mid s) > 0$ then $\mu(a \mid s) > 0$ for all (s,a) that can be visited; otherwise ratios contain division by zero.

4.6 Typical exam pitfalls

- MC is **model-free** and **does not bootstrap**; it uses complete returns.
- First-visit vs every-visit: every-visit gives more samples but within-episode samples are correlated.
- For control, MC is usually written in terms of $Q(s,a)$, not only $V(s)$.
- Off-policy: forgetting the **coverage** condition ($\mu = 0 \Rightarrow \pi = 0$) and the product form of ρ.
- Ordinary IS can be numerically unstable; weighted IS is the common practical default.

Temporal Difference Learning

5.1 Core idea: bootstrapping from experience (model-free)

Definition

Temporal-Difference (TD) learning is **model-free** learning of value functions from sampled experience that **bootstraps**: it updates estimates using other learned estimates. It sits between:

- **DP**: uses a model and expected backups;
- **MC**: uses sampled episodes and full returns (no bootstrapping);
- **TD**: uses sampled transitions and bootstraps from next-state values.

5.2 TD(0) prediction

Key Formula / Theorem

(**TD(0) target and TD error**) For a transition $S_t \to S_{t+1}$ with reward R_{t+1}, the **TD(0) target** for $v_\pi(S_t)$ is

$$R_{t+1} + \gamma V(S_{t+1}),$$

and the **TD error** is

$$\delta_t \;=\; R_{t+1} + \gamma V(S_{t+1}) - V(S_t).$$

Note

If $\gamma = 0$, TD(0) considers only immediate reward: the target is R_{t+1} and $\delta_t = R_{t+1} - V(S_t)$.

Key Formula / Theorem

(**TD(0) update**) With step-size $\alpha \in (0, 1]$:

$$V(S_t) \leftarrow V(S_t) + \alpha\,\delta_t \;=\; V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t)).$$

Only the current state value is updated per time step (tabular case).

Key Formula / Theorem

(**Convergence (tabular, on-policy TD(0))**) TD(0) prediction converges to v_π if (i) the policy is fixed and on-policy, (ii) all states are visited infinitely often, and (iii) step sizes satisfy Robbins–Monro conditions *per state*:

$$\sum_t \alpha_t(s) = \infty, \qquad \sum_t \alpha_t(s)^2 < \infty.$$

Typically assume $\gamma < 1$ for continuing tasks, or episodic tasks (terminal states) where γ may be 1. Off-policy TD and/or function approximation can diverge.

Example

If $V(S_t) = 2$, $R_{t+1} = -1$, $\gamma = 0.9$, $V(S_{t+1}) = 3$, then

$$\delta_t = -1 + 0.9 \cdot 3 - 2 = -0.3.$$

With $\alpha = 0.1$: $V(S_t) \leftarrow 2 + 0.1(-0.3) = 1.97$.

Intuition

(**Bias–variance intuition**) TD targets are typically **lower variance** (use one reward + a value estimate), but can be **more biased** early due to bootstrapping from imperfect V. MC targets are unbiased w.r.t. the return but can have high variance (especially with long episodes).

Note

(**TD vs DP (exam nuance)**) DP uses *expected* backups under the model; TD uses *sample* backups. In deterministic environments, the sampled successor equals the expectation. If TD(0) converges, it converges to v_π for the same γ. Step size α mainly affects speed/stability, not the fixed point (tabular on-policy prediction). Exact *sequences* of intermediate estimates can still differ depending on update order and whether you use two-array vs in-place updates.

5.3 TD control: learning action values

Key Formula / Theorem

For control it is common to learn **action values** $Q(s,a) \approx q_\pi(s,a)$ (or $q_*(s,a)$). The generic TD control update has the form

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big(\underbrace{R_{t+1} + \gamma\,\mathrm{Target}(S_{t+1})}_{\text{backup target}} - Q(S_t, A_t)\Big).$$

Key Formula / Theorem

(**Relating V and Q under a policy**) For any policy π,

$$v_\pi(s) = \sum_a \pi(a \mid s)\, q_\pi(s,a).$$

If π is deterministic, $v_\pi(s) = q_\pi(s, \pi(s))$.

5.3.1 SARSA (on-policy TD control)

Key Formula / Theorem

(**SARSA**) On-policy TD control: the target uses the *next action actually taken* by the *same* policy.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\Big).$$

Note

(**Action selection in SARSA**) In practice, SARSA is typically run with an ε-greedy policy w.r.t. the current estimate Q to ensure exploration. Because SARSA is **on-policy**, the *same* policy both (i) generates experience and (ii) defines the bootstrapping target via $A_{t+1} \sim \pi(\cdot \mid S_{t+1})$. If S_{t+1} is terminal, define $Q(S_{t+1}, \cdot) = 0$.

Intuition

(**Algorithm view**) SARSA updates after every transition, using the 5-tuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$. This is the control analogue of TD(0) prediction, but on (s, a)-pairs.

5.3.2 Off-policy SARSA and importance sampling (exam quick)

Note

(**Off-policy SARSA**) To learn about a target policy π while behaving according to μ, one common correction is importance sampling. With ratio

$$\rho_{t+1} = \frac{\pi(A_{t+1} \mid S_{t+1})}{\mu(A_{t+1} \mid S_{t+1})},$$

an off-policy SARSA(0) update is

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\, \rho_{t+1}\Big(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\Big).$$

Q-learning is off-policy and *does not* require importance sampling because its target uses $\max_a Q(S_{t+1}, a)$.

5.3.3 Expected SARSA (on-policy, lower variance target)

Key Formula / Theorem

(**Expected SARSA**) Replace the sampled $Q(S_{t+1}, A_{t+1})$ by its expectation under the policy π:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big(R_{t+1} + \gamma \sum_a \pi(a \mid S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t)\Big).$$

Often lower variance than SARSA.

Note

SARSA vs Expected SARSA (exam wording). Expected SARSA replaces the sampled next-action value $Q(S_{t+1}, A_{t+1})$ by its expectation under the policy. Conditioned on S_{t+1}, the Expected SARSA target equals the *expected* SARSA target:

$$\mathbb{E}[Q(S_{t+1}, A_{t+1}) \mid S_{t+1}] = \sum_a \pi(a \mid S_{t+1}) Q(S_{t+1}, a).$$

So SARSA moves in a *random* direction whose expectation matches Expected SARSA; it is not "deterministically" the same update.

5.3.4 Q-learning (off-policy TD control)

Key Formula / Theorem

(**Q-learning**) Off-policy TD control: the target uses a greedy action at the next state, independent of the behavior policy used to generate experience:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big(R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\Big).$$

Note

(**Behaviour vs target policy**) Q-learning is naturally described with two policies: the **behaviour** policy μ generates data and should explore (e.g. ε-greedy); the **target** policy π is the greedy policy $\pi(s) = \arg\max_a Q(s,a)$ that the update targets.

Key Formula / Theorem

(**Convergence (tabular Q-learning, sketch)**) Under standard conditions, (i) all state–action pairs (s,a) are updated infinitely often (ensured by an exploratory μ, e.g. ε-greedy), and (ii) step sizes satisfy Robbins–Monro conditions per (s,a), then Q converges to q_* with probability 1.

Note

Expected SARSA vs Q-learning. If the *target* policy is greedy/deterministic, the expectation collapses to a max, and Expected SARSA reduces to Q-learning:

$$\sum_a \pi(a \mid s')Q(s',a) = \max_a Q(s',a).$$

Note

(**Why no importance sampling correction (1-step)**) Although Q-learning is off-policy, its update conditions on the sampled (S_t, A_t). Given (S_t, A_t), the next state and reward are sampled from the environment dynamics $p(S_{t+1}, R_{t+1} \mid S_t, A_t)$ (not from μ or π). The off-policy aspect enters through the *greedy backup* $\max_a Q(S_{t+1}, a)$, so no action-probability ratio is needed for this one-step target.

Note

(**Maximization bias (overestimation)**) Using $\max_a Q(S_{t+1}, a)$ tends to *overestimate* action values in stochastic settings because it uses a maximum over noisy estimates. A standard fix is **Double Q-learning**.

Key Formula / Theorem

(**Double Q-learning**) Maintain two estimates Q_1, Q_2. For each transition $(S_t, A_t, R_{t+1}, S_{t+1})$, flip a fair coin:

$$\text{If heads:} \quad a^* = \arg\max_a Q_1(S_{t+1}, a), \qquad Q_1(S_t, A_t) \leftarrow Q_1(S_t, A_t) + \alpha\Big(R_{t+1} + \gamma\, Q_2(S_{t+1}, a^*) - Q_1(S_t, A_t)\Big).$$

$$\text{If tails:} \quad a^* = \arg\max_a Q_2(S_{t+1}, a), \qquad Q_2(S_t, A_t) \leftarrow Q_2(S_t, A_t) + \alpha\Big(R_{t+1} + \gamma\, Q_1(S_{t+1}, a^*) - Q_2(S_t, A_t)\Big).$$

The samples are split randomly between Q_1 and Q_2, which decorrelates them and reduces maximization bias.

Intuition

(**On-policy vs off-policy in one line**)

$$\begin{aligned} \text{SARSA: target uses } A_{t+1} &\sim \pi(\cdot \mid S_{t+1}) \quad \text{(on-policy)}, \\ \text{Q-learning: target uses } \arg\max_a Q(S_{t+1}, a) & \quad \text{(off-policy)}. \end{aligned}$$

5.4 n-step TD and the TD–MC spectrum

Key Formula / Theorem

(**n-step return for prediction**) The n-step TD target (state value) is

$$G_t^{(n)} \;=\; R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}).$$

Then update $V(S_t) \leftarrow V(S_t) + \alpha(G_t^{(n)} - V(S_t))$.

Intuition

(**Endpoints**) $n = 1$ gives TD(0). In episodic tasks, taking $n = T - t$ removes the bootstrap term and recovers MC (sometimes called TD(1) in this context).

Note

n-step TD can update *before* episode end: once n rewards are observed, you can update from the partial trajectory.

Note

If n is larger than the remaining episode length, the n-step target becomes the full return (MC), *not* DP.

5.5 Eligibility traces and TD(λ)

Key Formula / Theorem

(**Forward view: λ-return**) TD(λ) uses a geometrically weighted mixture of n-step returns:

$$G_t^{\lambda} \;=\; (1-\lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} G_t^{(n)} \;+\; \lambda^{T-t-1} G_t,$$

for episodic tasks (where G_t is the full MC return). Special cases: $\lambda = 0$ gives TD(0); $\lambda \to 1$ approaches MC.

Key Formula / Theorem

(**Backward view: eligibility traces, tabular prediction**) Maintain traces $e_t(s)$:

$$e_t(s) \;=\; \gamma\lambda\, e_{t-1}(s) + \mathbf{1}\{S_t = s\}.$$

Then update *all* states proportionally to their trace:

$$V(s) \leftarrow V(s) + \alpha\, \delta_t\, e_t(s) \quad \text{for all } s.$$

This implements the same target as the forward view (under standard conditions).

Key Formula / Theorem

(**Eligibility traces for action values**) Analogously,

$$e_t(s,a) = \gamma\lambda\, e_{t-1}(s,a) + \mathbf{1}\{S_t = s, A_t = a\}, \quad Q(s,a) \leftarrow Q(s,a) + \alpha\, \delta_t\, e_t(s,a),$$

with the TD error δ_t depending on SARSA / Expected SARSA / Q-learning target.

5.5.1 Batch learning and function approximation

Note

Batch TD vs batch MC (classic result). In the *tabular* case, batch TD(0) converges to the value function of the **maximum-likelihood Markov model** implied by the data (i.e., the DP solution on the empirical transition dynamics). Batch MC instead converges to the solution that best fits the observed *returns* (a regression / minimum squared error view). Therefore, they generally converge to *different* solutions.

Note

(**No general convergence guarantees**) With function approximation + bootstrapping (semi-gradient TD, TD(λ), etc.), convergence is not guaranteed in full generality (algorithms can diverge).

5.6 Past-exam questions

Exam-style question

What is an advantage of TD learning over Monte Carlo methods?

Key idea / answer sketch

TD updates online (after each step) and therefore does not need to wait for an episode to end; in practice this often yields faster learning.

Exam-style question

SARSA requires choosing a second action in the second state with the same policy as the first. True/False?

Key idea / answer sketch

True. SARSA is on-policy: it needs $A_{t+1} \sim \pi(\cdot \mid S_{t+1})$ to form the target $Q(S_{t+1}, A_{t+1})$.

Exam-style question

The TD(λ) algorithm computes a weighted average n-step TD backup target. True/False?

Key idea / answer sketch

True. The λ-return is a geometric mixture of n-step targets (forward view), implemented efficiently via eligibility traces (backward view).

5.7 Typical exam pitfalls

- Confusing **bootstrapping**: TD uses $V(S_{t+1})$ or $Q(S_{t+1}, \cdot)$ inside the target; MC does not.
- Mixing up **SARSA vs Q-learning**: SARSA uses the *actually taken* A_{t+1} (on-policy), Q-learning uses $\max_a$ (off-policy).
- Forgetting that Expected SARSA takes an **expectation under the policy** at S_{t+1}.
- Misremembering endpoints: $\lambda = 0$ gives TD(0); $\lambda \to 1$ approaches MC (episodic).
- Eligibility traces: they **decay** by $\gamma\lambda$ and assign **credit backward** to recently visited states (or state-action pairs).

Chapter 6

Policy Gradient Methods

6.1 Why policy gradients

Definition

Policy gradient (PG) methods optimize a **parameterized policy** $\pi_\theta(a \mid s)$ by directly maximizing an objective $J(\theta)$ using gradient ascent:

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta).$$

Unlike value-based control (e.g. Q-learning), PG learns the policy *explicitly*.

Intuition

Typical motivations:

- **Continuous actions:** $\arg\max_a Q(s,a)$ can be hard or ill-defined; π_θ can output continuous distributions.
- **Stochastic policies:** naturally represented (useful for exploration and for optimality in some settings).
- **Differentiable policies:** enable gradient-based optimization with function approximation.

Key Formula / Theorem

A common objective is the expected return from a start-state distribution:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\Big[\sum_{t=0}^{T-1} \gamma^t R_{t+1}\Big],$$

where $\tau = (S_0, A_0, R_1, S_1, \dots)$ is a trajectory induced by π_θ.

6.2 REINFORCE

Key Formula / Theorem

(**Score function trick / log-derivative identity**) For a differentiable probability distribution $\pi_\theta(a \mid s)$:

$$\nabla_\theta \pi_\theta(a \mid s) \;=\; \pi_\theta(a \mid s)\, \nabla_\theta \log \pi_\theta(a \mid s).$$

This is the key step to derive the REINFORCE estimator.

Key Formula / Theorem

(**Policy gradient theorem (one common form)**)

$$\nabla_\theta J(\theta) \propto \mathbb{E}_{\pi_\theta}\left[\sum_t \nabla_\theta \log \pi_\theta(A_t \mid S_t)\, q_{\pi_\theta}(S_t, A_t)\right].$$

Importantly, this does **not** require knowing the environment dynamics $p(s', r \mid s, a)$.

Key Formula / Theorem

(**REINFORCE: Monte Carlo policy gradient**) Using complete-episode returns, for each time step t in an episode:

$$\theta \leftarrow \theta + \alpha\, G_t\, \nabla_\theta \log \pi_\theta(A_t \mid S_t).$$

Intuition: increase probability of actions that led to high return.

Note

Properties:

- Unbiased gradient estimate (under standard assumptions), but can have **high variance**.
- **Episodic / MC**: must wait until (a segment of) the return is known.

Intuition

If an action A_t yielded a surprisingly good return G_t, then the update pushes θ in the direction that increases $\log \pi_\theta(A_t \mid S_t)$, i.e. increases $\pi_\theta(A_t \mid S_t)$. If G_t is negative, the update decreases that probability.

Note

REINFORCE is **on-policy**: trajectories must be generated by the current policy π_θ (or corrected with explicit off-policy machinery, which is not the standard REINFORCE algorithm).

Note

Course/exam convention. In this script, "REINFORCE" refers to the **episodic Monte Carlo** policy gradient that uses G_t from complete episodes. Continuing-task variants require additional machinery (e.g. average-reward formulations or actor–critic with TD-style critics), so the plain REINFORCE statement "effective for episodic and non-episodic MDPs" is **false** under this convention.

6.3 Baselines and variance reduction

Key Formula / Theorem

(**Baseline trick**) For any function $b(s)$ that does not depend on the action,

$$\mathbb{E}_{\pi_\theta}[b(S_t)\, \nabla_\theta \log \pi_\theta(A_t \mid S_t)] = 0.$$

Therefore you can replace G_t by $G_t - b(S_t)$ without changing the expectation of the gradient, but potentially reducing variance (baseline shifts all actions equally on average).

Key Formula / Theorem

(**REINFORCE with baseline**)

$$\theta \leftarrow \theta + \alpha\, (G_t - b(S_t))\, \nabla_\theta \log \pi_\theta(A_t \mid S_t).$$

Common choices:

- $b(s) = \bar{G}$ (running mean return),
- $b(s) = V_w(s)$, a learned state-value function (most common).

Note

REINFORCE with baseline vs actor–critic vs TD: If $b(s) = V_w(s)$ is learned, this can be viewed as an *actor + critic* setup. However, it is **not** a TD method unless the critic is trained with TD targets (e.g., using δ_t).

Definition

(**Advantage function**) If $b(s) = v_\pi(s)$, then $G_t - b(S_t)$ is an estimate of the **advantage**:

$$A_\pi(s, a) = q_\pi(s, a) - v_\pi(s).$$

Advantage-based updates typically have lower variance and clearer signal: "how much better than expected was this action?"

6.4 Actor–critic

6.4.1 (Very brief) PPO

Key Formula / Theorem

Proximal Policy Optimization (PPO) is a popular **actor–critic** method that stabilizes policy updates via a **clipped surrogate** objective. With probability ratio

$$r_t(\theta) = \frac{\pi_\theta(A_t \mid S_t)}{\pi_{\theta_{\text{old}}}(A_t \mid S_t)},$$

a common objective is

$$L^{\text{CLIP}}(\theta) = \mathbb{E}\left[\min(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)\right].$$

Clipping prevents the update from changing the policy too much in one step (stabilizes training).

Definition

Actor–critic methods combine:

- an **actor**: a parameterized policy $\pi_\theta(a \mid s)$,
- a **critic**: a value estimator (e.g. $V_w(s)$ or $Q_w(s, a)$).

The critic provides a learning signal (often a TD error) to update the actor.

Note

The actor update needs the critic only as a scalar signal (e.g., δ_t or $\hat{A}_t$); it does *not* require $\nabla_w V_w$. The gradient $\nabla_w V_w$ is used for updating the critic parameters w.

Key Formula / Theorem

(**TD error as advantage estimate**) With critic V_w,

$$\delta_t = R_{t+1} + \gamma V_w(S_{t+1}) - V_w(S_t)$$

is a one-step estimate of the advantage of taking A_t in S_t.

Key Formula / Theorem

(**Typical actor update**)

$$\theta \leftarrow \theta + \alpha\, \delta_t\, \nabla_\theta \log \pi_\theta(A_t \mid S_t).$$

Typical critic update (TD(0) on V_w):

$$w \leftarrow w + \beta\, \delta_t\, \nabla_w V_w(S_t).$$

Intuition

Key tradeoff: Compared to REINFORCE, actor–critic can update **online** (no need to wait for episode end) and usually reduces variance, but introduces **bias** due to bootstrapping / approximate critic.

6.5 Past-exam questions

Exam-style question

Why can we subtract a baseline $b(S_t)$ from the return in REINFORCE without biasing the gradient?

Key idea / answer sketch

Because the expected contribution of $b(S_t)\nabla_\theta \log \pi_\theta(A_t \mid S_t)$ under $A_t \sim \pi_\theta(\cdot \mid S_t)$ is zero, so it does not change $\mathbb{E}[\nabla_\theta J(\theta)]$.

Exam-style question

REINFORCE is unbiased but often high variance. True/False?

Key idea / answer sketch

True. It uses Monte Carlo returns; variance can be large without baselines / critics.

Exam-style question

Actor–critic methods can update before an episode ends. True/False?

Key idea / answer sketch

True. The critic supplies a TD-style signal (bootstrapping) for online updates.

6.6 Typical exam pitfalls

- Mixing up signs: PG is usually **gradient ascent** on $J(\theta)$.
- Forgetting the **log-derivative** form: updates use $\nabla_\theta \log \pi_\theta(A_t \mid S_t)$, not $\nabla_\theta \pi_\theta$ directly.
- Baseline must not depend on the chosen action (otherwise it can bias the gradient).
- Actor–critic: critic reduces variance but introduces bias; REINFORCE is unbiased but high variance.
- Confusing G_t (MC return) with δ_t (TD error / advantage estimate).
- With expert state–action pairs (s, a), you can pretrain a policy via supervised learning (maximize $\log \pi_\theta(a \mid s)$) before RL fine-tuning.